

Objetos y referencias en memoria

Python | Etapa 1

Guía de aprendizaje autónomo

Comprende qué cambia cuando ejecutas una instrucción: el estado de un objeto, una referencia o ambos. Aprende a predecir esos cambios, explicarlos y comprobarlos.

Lee y razona	Comprueba y corrige
10 temas progresivos 20 ejercicios de predicción 10 variaciones	10 ejemplos resueltos 4 prácticas de laboratorio Respuestas razonadas

Papel primero

Lee, predice y dibuja antes de abrir el intérprete. En las prácticas señaladas, ejecuta el código y compara el resultado con tu explicación. Conserva tus errores iniciales: te ayudan a localizar qué debes revisar.

Nombre:

Grupo:

Fecha:

Edición 1.0 | Septiembre de 2026

Formato carta, preparado para impresión en blanco y negro. Los programas del estudiante funcionan con Python 3.10 o posterior y su biblioteca estándar.

TU RECORRIDO

Avanza por comprensión

Sigue el orden. Avanza cuando puedas explicar el ejemplo y resolver una variación. Si una predicción falla, revisa las referencias antes de continuar.

Tema	Página
1.1 Objetos, tipos y mutabilidad	4
1.2 Variables, asignación y referencias	7
1.3 Identidad e igualdad	10
1.4 Mutación y reasignación	13
1.5 Referencias en funciones	18
1.6 Copias de listas simples	23
1.7 Anidamiento y copia superficial	26
1.8 Repetición de listas y matrices	29
1.9 Copia profunda	34
1.10 Integración y matices de mutabilidad	37

Momentos de ejecución

Práctica	Después de	Página
P01 Identidad y cambios de estado	1.4	15
P02 Modificar, reasignar y devolver	1.5	20
P03 Construir filas independientes	1.8	31
P04 Un equipo de trabajo independiente	1.10	39

Cierre y autoevaluación: página 42. Glosario y fuentes: página 43.

Cómo leer los diagramas

Las cajas representan objetos o nombres, según su etiqueta. Las flechas representan referencias. A, B y C son etiquetas simbólicas, nunca direcciones reales. Los valores dentro de las cajas de listas se abrevian; el dibujo no representa posiciones físicas contiguas de memoria.

ANTES DE EMPEZAR

Cómo trabajar con la guía

Se supone que ya reconoces variables, índices de listas, claves de diccionarios, ciclos sencillos y la sintaxis de una función. Los recordatorios de abajo te permiten empezar sin consultar una página web.

Acción	Qué haces
1. Leer	Estudia el concepto, el ejemplo resuelto y su diagrama.
2. Predecir	Escribe la salida y justifica qué objeto o referencia cambia.
3. Comprobar	En las prácticas señaladas, ejecuta y registra el resultado real.
4. Investigar	Sigue las flechas y corrige la explicación, conservando tu predicción.
5. Modificar	Cambia una sola condición y predice otra vez antes de ejecutar.

Recordatorio de sintaxis

Expresión	Lectura
<code>datos[0]</code>	Primer elemento de una lista. Los índices empiezan en 0.
<code>alumno["edad"]</code>	Valor asociado a la clave edad de un diccionario.
<code>lista.append(x)</code>	Agrega un elemento al final de la lista existente.
<code>lista[:]</code>	Con listas, obtiene una copia superficial de todas sus posiciones.
<code>def f(datos):</code>	Define una función cuyo parámetro local se llama datos.
<code>return resultado</code>	Devuelve un objeto al código que llamó a la función.
<code>range(3)</code>	Permite recorrer los valores 0, 1 y 2 en un ciclo.
<code>None</code>	Objeto que indica ausencia de un resultado útil en estos ejemplos.

Preparación para el laboratorio

Abre la carpeta `etapa_01_objetos_referencias` en la computadora. Cada archivo es independiente y debe ejecutarse desde el inicio. Ejemplo en macOS o Linux: `python3 ejercicios/e05.py`. En Windows: `py ejercicios/e05.py`. También puedes ejecutar el archivo completo desde tu editor.

No se necesitan servicios en línea ni paquetes adicionales. Mantén la guía impresa abierta y utiliza el intérprete para comprobar tus predicciones. En un cuaderno interactivo, reinicia el entorno antes de cada ejemplo para evitar estados anteriores.

Las identidades se comparan como `True` o `False`. No se piden valores numéricos de `id()` ni se califican direcciones de memoria.

LECTURA Y RAZONAMIENTO

1.1 Objetos, tipos y mutabilidad

Meta: Distinguir entre modificar un objeto y obtener un valor distinto, e identificar los tipos mutables habituales.

Si un número es inmutable, ¿por qué podemos escribir $n = n + 1$?

En Python los datos se representan mediante objetos. Un objeto tiene un tipo, un valor y una identidad. El tipo determina qué operaciones admite; el valor describe su contenido o estado. La identidad permite distinguirlo de otros objetos y la estudiaremos en 1.3.

Un objeto mutable admite cambios en su estado después de crearse. Un objeto inmutable conserva su estado. Esta propiedad corresponde al objeto: una variable puede pasar a nombrar otro objeto. Por eso $n = n + 1$ es válido aunque los enteros sean inmutables.

Mutables	Inmutables
list, dict, set	int, float, bool, str, tuple
Consulta adicional: bytearray	Consulta: complex, bytes, range, frozenset

Ejemplo resuelto D01

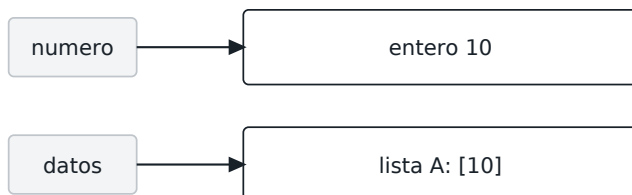
```
numero = 10
numero = numero + 1
datos = [10]
datos[0] = 11
print(numero)
print(datos)
```

Salida

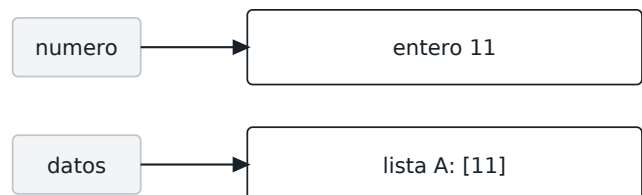
```
11
[11]
```

La suma obtiene el valor 11 y numero queda asociado al entero correspondiente. No modifica al entero 10. En cambio, `datos[0] = 11` cambia qué elemento ocupa la primera posición de la misma lista; no modifica al entero que antes estaba allí.

Antes de las operaciones



Después de las operaciones



Observa: Cambiar una variable no demuestra que el objeto anterior sea mutable. Una cadena admite operaciones como `upper()`, pero estas no modifican la cadena original.

RESUELVE EN PAPEL

1.1 Predice y explica

Escribe la salida antes de ejecutar. Conserva tu primera respuesta y usa cajas y flechas para justificarla. Los programas se reinician de forma independiente.

E01 | Una operación sobre una cadena

```
texto = "hola"  
mayusculas = texto.upper()  
print(texto)  
print(mayusculas)
```

1. Escribe las dos líneas de salida.
2. ¿Se modificó la cadena a la que se refiere texto? Explica.

Predicción, dibujo o explicación:

E02 | Modificar un diccionario

```
alumno = {"nombre": "Ana", "edad": 18}  
alumno["edad"] = 19  
print(alumno["nombre"])  
print(alumno["edad"])
```

1. Escribe la salida y señala qué objeto cambió.
2. Clasifica int, str, list, tuple, dict y set por mutabilidad.

Predicción, dibujo o explicación:

Una variación

En e01, sustituye la asignación a mayusculas por `texto = texto.upper()` y muestra solo texto. ¿Cambiaría la mutabilidad de str?

Predicción de la variación y motivo:

RESPUESTAS RAZONADAS | DESPUÉS DEL INTENTO

1.1 Comprueba tu razonamiento

Compara tus respuestas con las explicaciones. Si te equivocaste, conserva tu predicción inicial e identifica qué objeto o referencia interpretaste de otra manera.

E01 | Salida esperada

```
hola  
HOLA
```

`upper()` produce el resultado en mayúsculas; texto conserva su asociación con la cadena original. La operación no modifica esa cadena.

E02 | Salida esperada

```
Ana  
19
```

El diccionario es mutable: cambia el valor asociado a la clave edad. No se transforma el entero 18 en 19 ni cambia el valor asociado a nombre. Clasificación: int, str y tuple son inmutables; list, dict y set son mutables.

La variación

Se imprimiría HOLA. str seguiría siendo inmutable: texto quedaría asociado a la cadena resultante.

Antes de avanzar

Puedo explicar por qué asignar un nuevo valor a una variable no cambia la mutabilidad de su objeto anterior.

Marca tu situación: ☐ Puedo justificarlo. ☐ Necesito revisar un ejemplo.

Mi corrección o explicación con mis propias palabras:

Si la explicación aún no es clara, vuelve al diagrama del tema y sigue cada flecha desde la instrucción que produce el cambio.

LECTURA Y RAZONAMIENTO

1.2 Variables, asignación y referencias

Meta: Representar nombres que comparten un objeto y explicar por qué una asignación no lo copia.

Al escribir `y = x`, ¿aparece otra lista o aparece otro nombre para la misma lista?

Una variable es un nombre vinculado a un objeto. Llamaremos referencia a la relación que permite llegar a ese objeto. En `x = [1, 2, 3]` se crea una lista y `x` queda asociado a ella. En `y = x` se evalúa `x` y se vincula `y` al mismo objeto.

La asignación no copia automáticamente el objeto. Dos nombres que llegan al mismo objeto son alias. Si modificamos ese objeto mediante uno de ellos, el cambio se observa también a través del otro. Esto no exige que uno de los nombres se actualice o envíe datos al otro.

Ejemplo resuelto D02

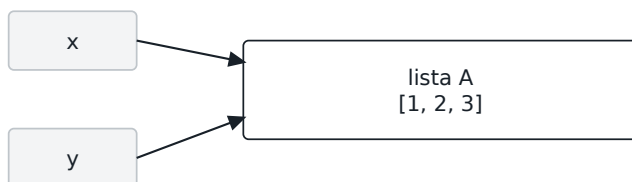
```
x = [1, 2, 3]
y = x
y.append(4)
print(x)
print(y)
```

Salida

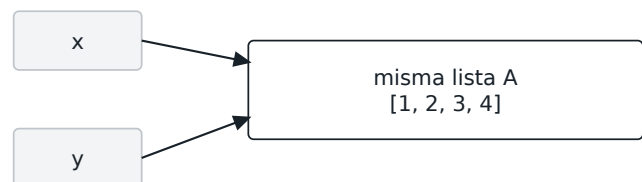
```
[1, 2, 3, 4]
[1, 2, 3, 4]
```

Después de `y = x` hay una sola lista. `append(4)` agrega un elemento a esa lista. Los dos `print` recorren el mismo objeto y muestran su estado actualizado.

Después de `y = x`



Después de `y.append(4)`



Observa: Un diagrama con dos nombres no implica dos objetos. Cuenta las cajas de objetos y sigue las flechas. Las etiquetas A y B de nuestros dibujos son simbólicas, nunca direcciones reales.

RESUELVE EN PAPEL

1.2 Predice y explica

Escribe la salida antes de ejecutar. Conserva tu primera respuesta y usa cajas y flechas para justificarla. Los programas se reinician de forma independiente.

E03 | Una tercera referencia

```
a = [7]
b = a
c = b
c.append(8)
print(a)
print(b)
print(c)
```

1. Predice la salida y dibuja los tres nombres.
2. ¿Cuántas listas se han creado?

Predicción, dibujo o explicación:

E04 | La misma asignación con enteros

```
a = 7
b = a
b = b + 1
print(a)
print(b)
```

1. Escribe la salida y dibuja las referencias al final.
2. ¿Se modificó el entero 7?

Predicción, dibujo o explicación:

Una variación

En e03, cambia solamente `b = a` por `b = [7]`. Mantén `c = b`. Predice las tres salidas y dibuja las listas que existirán.

Predicción de la variación y motivo:

RESPUESTAS RAZONADAS | DESPUÉS DEL INTENTO

1.2 Comprueba tu razonamiento

Compara tus respuestas con las explicaciones. Si te equivocaste, conserva tu predicción inicial e identifica qué objeto o referencia interpretaste de otra manera.

E03 | Salida esperada

```
[7, 8]  
[7, 8]  
[7, 8]
```

a, b y c se refieren a una única lista. `c.append(8)` la modifica. Las asignaciones `b = a` y `c = b` no crean listas adicionales.

E04 | Salida esperada

```
7  
8
```

Tras `b = a` ambos nombres se refieren al mismo entero. La última asignación vincula b al entero 8; a conserva su referencia al 7. La regla de asignación es la misma que con listas.

La variación

Las salidas serían `[7]`, `[7, 8]` y `[7, 8]`. a se refiere a una lista; b y c comparten una segunda lista creada por `[7]`.

Antes de avanzar

Puedo distinguir el número de nombres del número de listas y explicar por qué compartir una lista propaga la observación de sus cambios.

Marca tu situación: ☐ Puedo justificarlo. ☐ Necesito revisar un ejemplo.

Mi corrección o explicación con mis propias palabras:

Si la explicación aún no es clara, vuelve al diagrama del tema y sigue cada flecha desde la instrucción que produce el cambio.

LECTURA Y RAZONAMIENTO

1.3 Identidad e igualdad

Meta: Anticipar si dos expresiones se refieren al mismo objeto sin depender de números de identidad concretos.

¿Dos listas que imprimen lo mismo tienen que ser la misma lista?

Para las listas de estos ejemplos, `==` compara sus contenidos; `is` comprueba si ambas expresiones se refieren al mismo objeto. Dos listas distintas pueden ser iguales. Podemos compartir una lista mediante asignación o crear dos listas separadas con el mismo contenido.

`id(objeto)` devuelve un entero que identifica al objeto durante su vida. En CPython coincide con una dirección de memoria; eso es un detalle de esa implementación. Los valores concretos no son una respuesta fija entre ejecuciones. Aquí solo anticiparemos coincidencias o diferencias entre objetos que existen al mismo tiempo.

Ejemplo resuelto D03

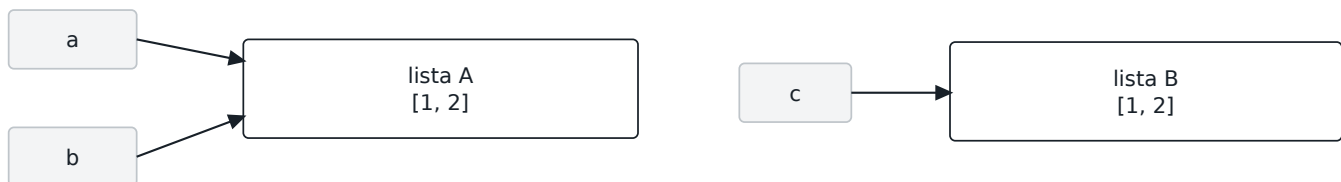
```
a = [1, 2]
b = a
c = [1, 2]
print(a == c)
print(a is c)
print(a is b)
print(id(a) == id(b))
```

Salida

```
True
False
True
True
```

`a` y `b` comparten una lista. `c` se refiere a otra lista con los mismos elementos. Por eso el contenido de `a` y `c` es igual, pero sus identidades son diferentes. Comparar los `id` de `a` y `b` confirma que se trata del mismo objeto.

Dos listas distintas; `a` y `b` comparten A



Observa: Usa `==` para comparar valores. No bases una regla general en `is` con números o cadenas literales: Python puede reutilizar ciertos objetos inmutables. Un `id` puede reutilizarse cuando el objeto anterior ya no existe.

RESUELVE EN PAPEL

1.3 Predice y explica

Escribe la salida antes de ejecutar. Conserva tu primera respuesta y usa cajas y flechas para justificarla. Los programas se reinician de forma independiente.

E05 | Tres comparaciones

```
a = [5]
b = [5]
c = a
print(a == b)
print(a is b)
print(b is c)
print(id(a) == id(c))
```

1. Predice cada booleano; no escribas direcciones.
2. Dibuja las listas y escribe qué nombres comparten identidad.

Predicción, dibujo o explicación:

E06 | Objetos vacíos

```
x = []
y = x
z = []
print(x == z)
print(x is y)
print(id(y) == id(z))
```

1. Escribe la salida.
2. ¿Sería válido exigir un número específico como respuesta a `id(x)`?

Predicción, dibujo o explicación:

Una variación

En e05 cambia únicamente `c = a` por `c = b`. ¿Cuáles de las cuatro respuestas cambian?

Predicción de la variación y motivo:

RESPUESTAS RAZONADAS | DESPUÉS DEL INTENTO

1.3 Comprueba tu razonamiento

Compara tus respuestas con las explicaciones. Si te equivocaste, conserva tu predicción inicial e identifica qué objeto o referencia interpretaste de otra manera.

E05 | Salida esperada

True
False
False
True

a y c llegan a una lista; b llega a otra. Las listas tienen el mismo contenido. La última comparación es True porque a y c identifican al mismo objeto vivo.

E06 | Salida esperada

True
True
False

Cada expresión `[]` crea una lista nueva. x y z son listas vacías iguales; y comparte la de x. Las dos listas están vivas y sus id son distintos.

La variación

La salida pasa a True, False, True, False. Cambian la tercera y la cuarta comparación. No es válido exigir un id numérico fijo en e06: lo relevante es la relación de identidad.

Antes de avanzar

Puedo justificar cada comparación siguiendo referencias, sin memorizar números de id ni confundir `==` con `is`.

Marca tu situación: ☐ Puedo justificarlo. ☐ Necesito revisar un ejemplo.

Mi corrección o explicación con mis propias palabras:

Si la explicación aún no es clara, vuelve al diagrama del tema y sigue cada flecha desde la instrucción que produce el cambio.

LECTURA Y RAZONAMIENTO

1.4 Mutación y reasignación

Meta: Identificar el objeto afectado por una operación y distinguir cambios de contenido de cambios de asociación.

¿Por qué `y.append(4)` y `y = y + [4]` pueden producir efectos diferentes sobre `x`?

Mutar es modificar el estado de un objeto existente. Reasignar un nombre es vincularlo a otro objeto. Una lista admite mutación mediante `append()`, `clear()` y asignaciones por índice. En cambio, `y = []` cambia la asociación del nombre `y`.

Con listas incorporadas, `+` crea una lista nueva y `+=` extiende la lista existente. Con enteros, `+=` obtiene un resultado y reasigna el nombre. No memorices `+=` como sinónimo universal de una de estas conductas: identifica el tipo y la operación.

Ejemplo resuelto D04

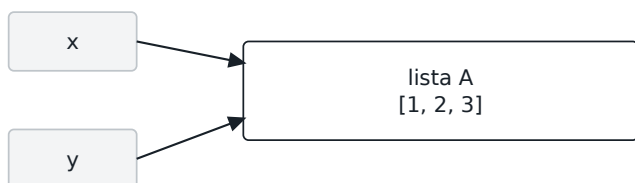
```
x = [1, 2, 3]
y = x
y = y + [4]
print(x)
print(y)
print(x is y)
```

Salida

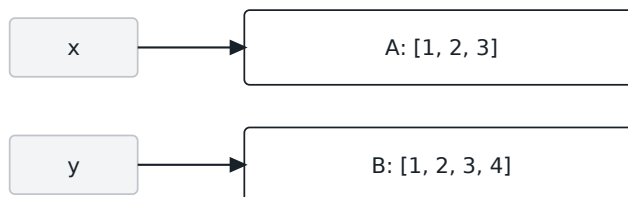
```
[1, 2, 3]
[1, 2, 3, 4]
False
```

La expresión `y + [4]` construye otra lista. La asignación mueve la referencia de `y` a esa lista. `x` continúa asociado a la primera. En `y[0] = 9`, en cambio, el nombre `y` no se reasigna: cambia un elemento de la lista a la que llega.

Antes de la concatenación



Después de `y = y + [4]`



Observa: `x.clear()` vacía una lista existente; `x = []` vincula `x` a una lista nueva. Además, `append()` y `clear()` devuelven `None`: escribir `x = x.append(4)` hace perder a `x` su referencia a la lista.

RESUELVE EN PAPEL

1.4 Predice y explica

Escribe la salida antes de ejecutar. Conserva tu primera respuesta y usa cajas y flechas para justificarla. Los programas se reinician de forma independiente.

E07 | La suma aumentada sobre una lista

```
x = [1, 2]
y = x
y += [3]
print(x)
print(y)
print(x is y)
```

1. Predice las tres salidas.
2. Contrasta este caso con el ejemplo resuelto de la sección.

Predicción, dibujo o explicación:

E08 | Vaciar y después reasignar

```
x = [1, 2]
y = x
y.clear()
y = [9]
print(x)
print(y)
print(x is y)
```

1. Dibuja el estado tras `clear()` y el estado final.
2. Identifica cuál línea muta y cuál reasigna.

Predicción, dibujo o explicación:

Una variación

Sustituye `y += [3]` por `y = y + [3]` en e07. Después compara `a = 10; b = a; b += 3`. Explica ambos casos.

Predicción de la variación y motivo:

COMPRUEBA EN PYTHON | PRÁCTICA DE LABORATORIO

P01 Identidad y cambios de estado

Meta: Comprobar la diferencia entre extender una lista compartida, construir otra lista y reasignar un entero.

Archivo: practicas/p01.py. Trabaja desde la carpeta etapa_01_objetos_referencias.

Programa inicial: primero predice

```
x = [1, 2]
y = x
y += [3]
a = 10
b = a
b += 3
print(x)
print(x is y)
print(a, b)
```

1. Escribe la salida prevista y dibuja las referencias antes de ejecutar el programa inicial.
2. Ejecuta el archivo y conserva tu predicción. Registra cualquier diferencia y su explicación.
3. Variante A: cambia solo `y += [3]` por `y = y + [3]`. Predice y ejecuta de nuevo desde el inicio.
4. Variante B: haz que `x` y `y` terminen vacías y sigan siendo la misma lista. Decide entre `y.clear()` y `y = []`. Comprueba tu elección.

Registro de comprobación

Puedes ampliar este registro en tu cuaderno. Cada variante se ejecuta desde el estado inicial.

Caso	Predicción	Observación y explicación
Inicial		
Variante / corrección		

Para ejecutar: `python3 practicas/p01.py` | En Windows: `py practicas/p01.py`

RESPUESTAS RAZONADAS | DESPUÉS DEL INTENTO

1.4 Comprueba tu razonamiento

Compara tus respuestas con las explicaciones. Si te equivocaste, conserva tu predicción inicial e identifica qué objeto o referencia interpretaste de otra manera.

E07 | Salida esperada

```
[1, 2, 3]
[1, 2, 3]
True
```

Para list, += extiende la lista existente. Los dos nombres siguen llegando a ella; por eso cambia lo que se observa desde x y la identidad compartida se conserva.

E08 | Salida esperada

```
[]
[9]
False
```

clear() primero vacía la lista compartida. Después y = [9] crea otra lista y cambia únicamente la asociación de y. x sigue llegando a la lista vacía.

La variación

La variante de listas imprime [1, 2], [1, 2, 3] y False: + construye una lista nueva. Con enteros, a sigue siendo 10 y b queda asociado al 13. El entero 10 no se modifica.

Antes de avanzar

Puedo señalar si una instrucción cambia un objeto, una posición del contenedor o la asociación de un nombre.

Marca tu situación: ☐ Puedo justificarlo. ☐ Necesito revisar un ejemplo.

Mi corrección o explicación con mis propias palabras:

Si la explicación aún no es clara, vuelve al diagrama del tema y sigue cada flecha desde la instrucción que produce el cambio.

SOLUCIÓN DE REFERENCIA | DESPUÉS DEL INTENTO

P01 Revisión de la práctica

Inicial: x queda [1, 2, 3], x is y es True y los enteros se muestran como 10 13.

Variante A: se imprime [1, 2], False y 10 13. + construye otra lista y la asignación cambia la referencia de y.

Variante B: clear() satisface las dos condiciones. Para comprobar contenidos e identidad también puedes imprimir y y comparar `id(x) == id(y)`; el booleano debe ser True.

Solución de la variante B

```
x = [1, 2]
y = x
y.clear()
a = 10
b = a
b += 3
print(x)
print(x is y)
print(a, b)
```

Salida

```
[]
True
10 13
```

clear() vacía el objeto compartido. `y = []` no cumpliría la condición: dejaría x intacta y separaría los nombres.

Archivo de referencia: `soluciones/s01.py`.

¿Qué cambié y qué evidencia demuestra que mi solución cumple la consigna?

LECTURA Y RAZONAMIENTO

1.5 Referencias en funciones

Meta: Explicar los efectos de mutar un argumento o reasignar un parámetro local, y recuperar resultados mediante return.

¿Por qué una función puede agregar elementos a tu lista, pero `lst = []` no la vacía?

Al llamar una función, sus parámetros se vinculan a los objetos recibidos. Python pasa argumentos por asignación: el parámetro es un nombre local que puede compartir objeto con un nombre del código que llama. La regla es la misma para listas, diccionarios, enteros y cadenas.

Mutar un objeto compartido permite observar el cambio desde fuera de la función. Reasignar el parámetro cambia solo su asociación local. Si la función obtiene otro objeto que el llamador necesita, puede devolverlo con return; el llamador decide a qué nombre asociarlo.

Ejemplo resuelto D05

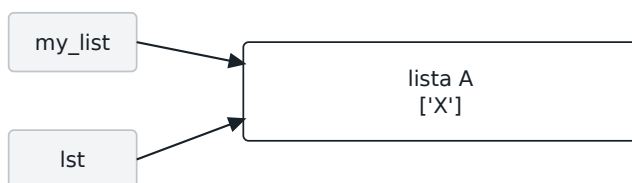
```
def add_item(lst):  
    lst.append("X")  
  
def reset_list(lst):  
    lst = []  
  
my_list = []  
add_item(my_list)  
reset_list(my_list)  
print(my_list)
```

Salida

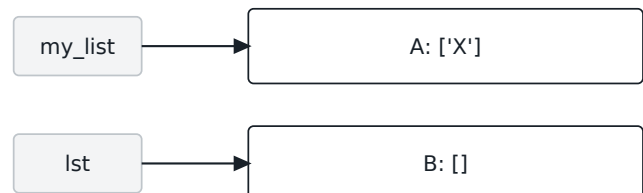
['X']

`add_item` vincula `lst` a la lista de `my_list` y la modifica. En `reset_list`, `lst` inicialmente llega a esa misma lista, pero `lst = []` lo vincula a otra. `my_list` conserva su referencia. Al acabar cada llamada, el nombre local `lst` no se convierte en un nombre del llamador.

Al entrar en `reset_list`



Durante `reset_list`, tras `lst = []`



Observa: El comportamiento no significa que las listas se pasen de una forma y los enteros de otra. Cambia lo que admite cada objeto. Una función sin return explícito devuelve None.

RESUELVE EN PAPEL

1.5 Predice y explica

Escribe la salida antes de ejecutar. Conserva tu primera respuesta y usa cajas y flechas para justificarla. Los programas se reinician de forma independiente.

E09 | Reasignar dentro de la función

```
def modify(lst):  
    lst = lst + [99]  
  
numbers = [1, 2, 3]  
modify(numbers)  
print(numbers)
```

1. Escribe la salida y dibuja las dos listas durante la llamada.
2. ¿Qué cambiaría si la función utilizara `lst.append(99)`?

Predicción, dibujo o explicación:

E10 | Actualizar y reiniciar un diccionario

```
def update_dict(d):  
    d["key"] = "value"  
  
def reset_dict(d):  
    d = {}  
  
my_dict = {}  
update_dict(my_dict)  
reset_dict(my_dict)  
print(my_dict)
```

1. Identifica la mutación y la reasignación local.
2. Propón una línea que permita a `reset_dict` vaciar el diccionario recibido.

Predicción, dibujo o explicación:

Una variación

En e09, haz que la función devuelva `lst + [99]`. Compara llamar `modify(numbers)` sin guardar el resultado y escribir `numbers = modify(numbers)`.

Predicción de la variación y motivo:

COMPRUEBA EN PYTHON | PRÁCTICA DE LABORATORIO

P02 Modificar, reasignar y devolver

Meta: Controlar si una función cambia la lista recibida o entrega otra lista al llamador.

Archivo: practicas/p02.py. Trabaja desde la carpeta etapa_01_objetos_referencias.

Programa inicial: primero predice

```
def agregar_final(datos):
    datos = datos + [99]

original = [1, 2]
resultado = agregar_final(original)
print(original)
print(resultado)
```

- 1. Predice original y resultado. Distingue las listas a las que llegan original y datos justo después de la asignación dentro de la función.
- 2. Variante A: cambia el cuerpo por datos.append(99), sin añadir return. Predice original y resultado.
- 3. Variante B: conserva original intacta y devuelve una lista con 99 al final. Guarda el retorno en resultado y compara su identidad con original.
- 4. En la variante B, cambia la llamada a agregar_final(original) sin asignarla. Explica qué lista continúa disponible mediante original.

Registro de comprobación

Puedes ampliar este registro en tu cuaderno. Cada variante se ejecuta desde el estado inicial.

Caso	Predicción	Observación y explicación
Inicial		
Variante / corrección		

Para ejecutar: python3 practicas/p02.py | En Windows: py practicas/p02.py

RESPUESTAS RAZONADAS | DESPUÉS DEL INTENTO

1.5 Comprueba tu razonamiento

Compara tus respuestas con las explicaciones. Si te equivocaste, conserva tu predicción inicial e identifica qué objeto o referencia interpretaste de otra manera.

E09 | Salida esperada

```
[1, 2, 3]
```

lst + [99] construye otra lista. La asignación solo vincula el parámetro local a ella; numbers sigue asociado a la lista inicial.

E10 | Salida esperada

```
{'key': 'value'}
```

update_dict cambia el diccionario compartido. reset_dict solo reasigna su parámetro d. Al retornar, my_dict sigue llegando al diccionario con la entrada key.

La variación

Sin guardar el retorno, numbers conserva [1, 2, 3]. Al asignarlo, numbers queda asociado a [1, 2, 3, 99]. La variante append(99) mutaría la lista original. En e10, d.clear() la vaciaría sin reasignar d.

Antes de avanzar

Puedo distinguir la modificación de un argumento de la reasignación de su parámetro y explicar cuándo debo usar el resultado devuelto.

Marca tu situación: ☐ Puedo justificarlo. ☐ Necesito revisar un ejemplo.

Mi corrección o explicación con mis propias palabras:

Si la explicación aún no es clara, vuelve al diagrama del tema y sigue cada flecha desde la instrucción que produce el cambio.

SOLUCIÓN DE REFERENCIA | DESPUÉS DEL INTENTO

P02 Revisión de la práctica

Inicial: aparecen [1, 2] y None. Justo después de reasignar datos, original llega a [1, 2] y el parámetro local datos llega a otra lista, [1, 2, 99].

Variante A: original pasa a [1, 2, 99]; resultado sigue siendo None porque append() no añade un return a la función.

Variante B: el código de solución devuelve otra lista. Si se ignora el retorno, original conserva [1, 2] y no se guarda un nombre para la lista resultante.

Solución de la variante B

```
def agregar_final(datos):  
    return datos + [99]  
  
original = [1, 2]  
resultado = agregar_final(original)  
print(original)  
print(resultado)  
print(original is resultado)
```

Salida

```
[1, 2]  
[1, 2, 99]  
False
```

return entrega la lista nueva al llamador. resultado queda asociado a ella y original conserva la primera.

Archivo de referencia: soluciones/s02.py.

¿Qué cambié y qué evidencia demuestra que mi solución cumple la consigna?

LECTURA Y RAZONAMIENTO

1.6 Copias de listas simples

Meta: Distinguir una asignación de una copia superficial y comprobar qué cambios quedan aislados en listas simples.

¿Qué diferencia existe entre `copia = original` y `copia = original.copy()`?

`original.copy()` crea otra lista. Esa nueva lista comienza con referencias a los mismos elementos que la primera. Se llama copia superficial: se crea el contenedor exterior, pero no se copian recursivamente los objetos que contiene.

Para listas, `original[:]` también crea una copia superficial. Si sus elementos son enteros, cadenas o booleanos, sustituir una posición en la copia no cambia el contenedor original. Esto no transforma los enteros ni las cadenas; cambia una referencia dentro de la lista nueva.

Ejemplo resuelto D06

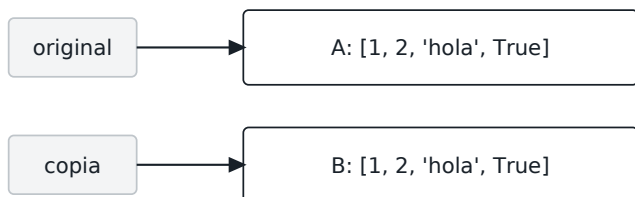
```
original = [1, 2, "hola", True]
copia = original.copy()
copia[0] = 99
print(original)
print(copia)
print(original is copia)
```

Salida

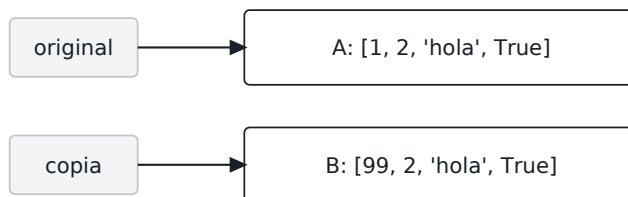
```
[1, 2, 'hola', True]
[99, 2, 'hola', True]
False
```

Se crean dos listas distintas. Cambiar la posición 0 de copia no altera la posición 0 de original. Las posiciones restantes pueden seguir refiriéndose a los mismos objetos inmutables; eso no impide que las listas sean distintas.

Después de `.copy()`



Después de `copia[0] = 99`



Observa: El nombre `copia` no garantiza una copia. `copia = original` sigue siendo una asignación. Tampoco basta decir 'los elementos son inmutables' si un elemento es una tupla que contiene una lista: revisaremos ese caso en 1.10.

RESUELVE EN PAPEL

1.6 Predice y explica

Escribe la salida antes de ejecutar. Conserva tu primera respuesta y usa cajas y flechas para justificarla. Los programas se reinician de forma independiente.

E11 | Una copia y un alias

```
original = [10, 20]
copia = original.copy()
alias = original
copia.append(30)
alias[0] = 99
print(original)
print(copia)
```

1. Predice la salida y dibuja las dos listas.
2. ¿Qué nombres comparten identidad?

Predicción, dibujo o explicación:

E12 | Copiar mediante un corte

```
a = ["A", "B"]
b = a[:]
print(a == b)
print(a is b)
b.clear()
print(a)
print(b)
```

1. Escribe cada línea de salida.
2. ¿Por qué vaciar b no vacía a?

Predicción, dibujo o explicación:

Una variación

En e12 sustituye `b = a[:]` por `b = a`. Predice nuevamente todos los resultados.

Predicción de la variación y motivo:

RESPUESTAS RAZONADAS | DESPUÉS DEL INTENTO

1.6 Comprueba tu razonamiento

Compara tus respuestas con las explicaciones. Si te equivocaste, conserva tu predicción inicial e identifica qué objeto o referencia interpretaste de otra manera.

E11 | Salida esperada

```
[99, 20]  
[10, 20, 30]
```

copia es otra lista y recibe el 30. alias comparte la lista de original y cambia su primera posición. Cada modificación afecta un contenedor distinto.

E12 | Salida esperada

```
True  
False  
['A', 'B']  
[]
```

a[:] construye otra lista con los mismos elementos. Inicialmente los contenidos coinciden. clear() vacía únicamente la lista a la que se refiere b.

La variación

Se imprimirían True, True, [] y []. b sería un alias de a y clear() vaciaría la única lista. En e11, original y alias comparten identidad.

Antes de avanzar

Puedo elegir entre compartir una lista y crear otra lista cuando necesito modificar su contenedor por separado.

Marca tu situación: ☐ Puedo justificarlo. ☐ Necesito revisar un ejemplo.

Mi corrección o explicación con mis propias palabras:

Si la explicación aún no es clara, vuelve al diagrama del tema y sigue cada flecha desde la instrucción que produce el cambio.

LECTURA Y RAZONAMIENTO

1.7 Anidamiento y copia superficial

Meta: Seguir referencias por varios niveles y distinguir la sustitución de una fila de la mutación de esa fila.

¿Por qué una copia distinta puede seguir compartiendo las listas que hay en su interior?

Una lista puede contener referencias a otras listas. `original[0]` llega al objeto que ocupa su primera posición; `original[0][0]` llega, a través de él, a una posición del contenedor interior. Conviene dibujar por separado la lista exterior y cada lista interior.

Una copia superficial crea otro contenedor exterior y reutiliza las referencias de sus elementos. Por eso `copia[0] = [99]` sustituye una posición del exterior nuevo, mientras `copia[0][0] = 99` modifica una lista interior que puede seguir compartida con original.

Ejemplo resuelto D07

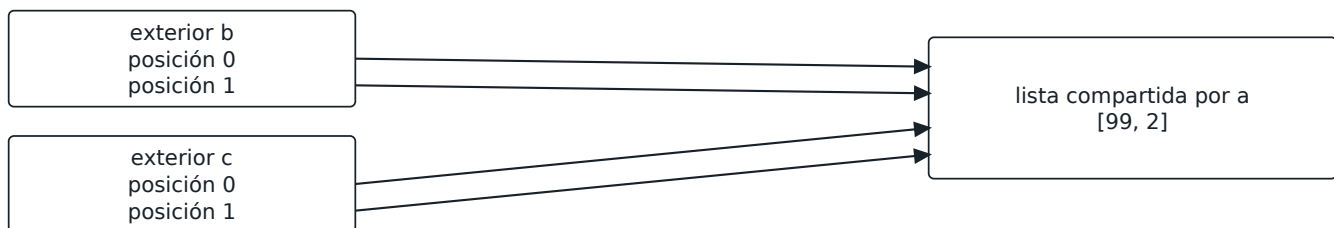
```
a = [1, 2]
b = [a, a]
c = b.copy()
c[0][0] = 99
print(a)
print(b)
print(c)
print(b is c)
```

Salida

```
[99, 2]
[[99, 2], [99, 2]]
[[99, 2], [99, 2]]
False
```

`b` y `c` son listas exteriores distintas. Sus cuatro posiciones llegan a la misma lista `a`. Modificar `c[0][0]` modifica esa única lista interior. Imprimir `b` y `c` produce el mismo contenido, aunque `b is c` sea `False`.

Tras `c[0][0] = 99`: exteriores distintos, una lista interior



Observa: Preguntar solo '¿se hizo una copia?' es insuficiente. Hay que identificar qué objeto se copió y cuáles continúan compartidos.

RESUELVE EN PAPEL

1.7 Predice y explica

Escribe la salida antes de ejecutar. Conserva tu primera respuesta y usa cajas y flechas para justificarla. Los programas se reinician de forma independiente.

E13 | Sustituir una fila de la copia

```
original = [[1, 2], [3, 4]]
copia = original.copy()
copia[0] = [99, 2]
print(original)
print(copia)
print(original[1] is copia[1])
```

1. Escribe la salida y señala qué referencia se sustituyó.
2. ¿Sigue existiendo alguna fila compartida?

Predicción, dibujo o explicación:

E14 | Modificar una fila compartida

```
original = [[1, 2], [3, 4]]
copia = original.copy()
copia[0][0] = 99
print(original)
print(copia)
print(original[0] is copia[0])
```

1. Predice las salidas y dibuja ambos niveles.
2. Explica por qué e13 y e14 afectan de manera distinta a original.

Predicción, dibujo o explicación:

Una variación

En e14 cambia solo `copia[0][0] = 99` por `copia.append([5, 6])`. ¿Qué contenedor se modifica?

Predicción de la variación y motivo:

RESPUESTAS RAZONADAS | DESPUÉS DEL INTENTO

1.7 Comprueba tu razonamiento

Compara tus respuestas con las explicaciones. Si te equivocaste, conserva tu predicción inicial e identifica qué objeto o referencia interpretaste de otra manera.

E13 | Salida esperada

```
[[1, 2], [3, 4]]  
[[99, 2], [3, 4]]  
True
```

La asignación cambia la primera posición del exterior copia. La primera fila original no se modifica. La segunda fila sigue siendo un objeto compartido entre los exteriores.

E14 | Salida esperada

```
[[99, 2], [3, 4]]  
[[99, 2], [3, 4]]  
True
```

La primera fila es la misma lista en ambos exteriores. Modificar su posición 0 se observa desde los dos. El efecto difiere de e13 porque cambia el objeto interior, no la referencia a la fila en el exterior.

La variación

Se modifica únicamente el exterior copia: original conserva `[[1, 2], [3, 4]]` y copia queda `[[1, 2], [3, 4], [5, 6]]`. La comparación de la primera fila sigue siendo `True`.

Antes de avanzar

Puedo seguir cada índice hasta el objeto afectado y separar la identidad del exterior de la identidad de sus elementos.

Marca tu situación: ☐ Puedo justificarlo. ☐ Necesito revisar un ejemplo.

Mi corrección o explicación con mis propias palabras:

Si la explicación aún no es clara, vuelve al diagrama del tema y sigue cada flecha desde la instrucción que produce el cambio.

LECTURA Y RAZONAMIENTO

1.8 Repetición de listas y matrices

Meta: Explicar qué referencias repite el operador `*` y construir matrices con filas independientes.

¿Por qué modificar una sola celda puede cambiar lo que vemos en las tres filas?

La repetición de una lista produce otra lista que repite las referencias a sus elementos. No clona esos elementos. En `[0] * 3` se obtiene una lista con tres posiciones referidas al entero 0. Sustituir una posición cambia esa lista; no modifica al entero 0.

En `[[0] * 3] * 3` se crea una sola fila `[0, 0, 0]`. La lista exterior repite tres veces la referencia a esa fila mutable. Para obtener filas independientes debemos construir una fila nueva en cada iteración, por ejemplo con `[[0] * 3 for _ in range(3)]`.

Ejemplo resuelto D08

```
matrix = [[0] * 3] * 3
matrix[0][0] = 1
print(matrix)
print(matrix[0] is matrix[1])
```

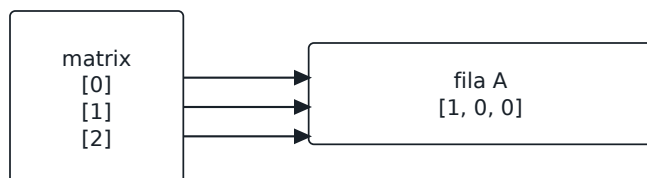
Salida

```
[[1, 0, 0], [1, 0, 0], [1, 0, 0]]
```

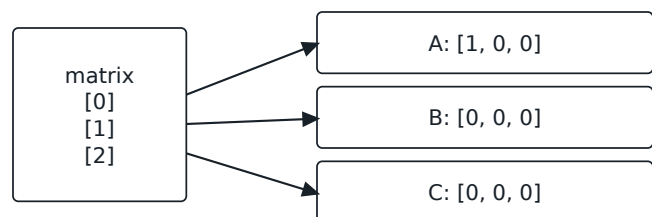
```
True
```

Hay una lista exterior y una fila interior. `matrix[0][0]` modifica esa fila interior; las tres posiciones del exterior siguen llegando a ella. Compartir puede ser intencional, pero no sirve si el problema exige que cada fila cambie por separado.

Con repetición: una fila compartida



Con filas construidas por separado



Observa: La comprensión evalúa `[0] * 3` en cada iteración; `_` es un nombre de variable cuyo valor no necesitamos. Usar `a = [0]` y `[a * 3] * 3` mantiene el mismo problema: `a * 3` crea una única fila nueva que luego se comparte.

RESUELVE EN PAPEL

1.8 Predice y explica

Escribe la salida antes de ejecutar. Conserva tu primera respuesta y usa cajas y flechas para justificarla. Los programas se reinician de forma independiente.

E15 | Una variable explícita

```
a = [0]
matrix = [a * 3] * 3
matrix[0][0] = 1
print(a)
print(matrix)
print(matrix[0] is matrix[2])
```

1. Predice las tres salidas.
2. Distingue la lista a de la fila producida por a * 3.

Predicción, dibujo o explicación:

E16 | Construcción con un ciclo

```
matrix = []
for _ in range(3):
    matrix.append([0] * 3)
matrix[0][0] = 1
print(matrix)
print(matrix[0] is matrix[1])
```

1. Predice la salida y dibuja las tres filas.
2. Reescribe la construcción mediante una comprensión de listas.

Predicción, dibujo o explicación:

Una variación

En e15 sustituye la construcción por `matrix = [(a * 3).copy() for _ in range(3)]`. ¿Sería indispensable `.copy()` aquí? ¿Qué tamaño tendría `[a.copy(), a.copy(), a.copy()]`?

Predicción de la variación y motivo:

COMPRUEBA EN PYTHON | PRÁCTICA DE LABORATORIO

P03 Construir filas independientes

Meta: Distinguir la copia del exterior de la creación de filas independientes.

Archivo: practicas/p03.py. Trabaja desde la carpeta etapa_01_objetos_referencias.

Programa inicial: primero predice

```
matrix = [[0] * 3] * 3
copia = matrix.copy()
copia[0][0] = 1
print(matrix)
print(copia)
print(matrix is copia)
print(matrix[0] is copia[0])
```

1. Dibuja ambos exteriores y sus filas. Predice las cuatro salidas y luego ejecuta.
2. Variante A: construye matrix con un ciclo que cree una fila nueva por iteración. Conserva copia = matrix.copy(). ¿Qué compartición permanece?
3. Variante B: además de construir filas independientes, copia cada fila para que cambiar una celda de copia no cambie matrix. Usa lo aprendido hasta 1.8.
4. Comprueba la independencia con is entre filas y entre cada fila original y su copia. No utilices valores numéricos fijos de id().

Registro de comprobación

Puedes ampliar este registro en tu cuaderno. Cada variante se ejecuta desde el estado inicial.

Caso	Predicción	Observación y explicación
Inicial		
Variante / corrección		

Para ejecutar: python3 practicas/p03.py | En Windows: py practicas/p03.py

RESPUESTAS RAZONADAS | DESPUÉS DEL INTENTO

1.8 Comprueba tu razonamiento

Compara tus respuestas con las explicaciones. Si te equivocaste, conserva tu predicción inicial e identifica qué objeto o referencia interpretaste de otra manera.

E15 | Salida esperada

```
[0]
[[1, 0, 0], [1, 0, 0], [1, 0, 0]]
True
```

`a * 3` crea una fila nueva distinta de `a`. El exterior comparte esa fila tres veces. La fila cambia, pero `a` sigue siendo la lista `[0]`. Al final son alcanzables tres listas mediante `a` y `matrix`: `a`, la fila y el exterior.

E16 | Salida esperada

```
[[1, 0, 0], [0, 0, 0], [0, 0, 0]]
False
```

En cada iteración se crea una fila nueva. El exterior termina con referencias a tres filas distintas. Sustituir una celda de la primera fila no modifica las otras dos.

La variación

Se obtienen filas independientes; `.copy()` es redundante porque `a * 3` ya se evalúa y crea una fila nueva en cada iteración. El ciclo de `e16` equivale a `[[0] * 3 for _ in range(3)]`. Con `a = [0]`, las tres copias de `a` formarían una matriz de 3 filas y 1 columna, no de 3 por 3.

Antes de avanzar

Puedo distinguir repetir referencias de construir objetos nuevos y comprobar la independencia entre filas.

Marca tu situación: ☐ Puedo justificarlo. ☐ Necesito revisar un ejemplo.

Mi corrección o explicación con mis propias palabras:

Si la explicación aún no es clara, vuelve al diagrama del tema y sigue cada flecha desde la instrucción que produce el cambio.

SOLUCIÓN DE REFERENCIA | DESPUÉS DEL INTENTO

P03 Revisión de la práctica

Inicial: se observan tres filas [1, 0, 0] en cada exterior; los exteriores son distintos, pero todos sus elementos llegan a una fila compartida.

Variante A: matrix y copia muestran [[1, 0, 0], [0, 0, 0], [0, 0, 0]]. Las filas de matrix son distintas entre sí, pero cada fila sigue compartida con la posición correspondiente de copia.

Variante B: la solución separa exteriores y filas. Copiar cada fila una vez es suficiente para estas celdas enteras; con objetos mutables dentro de las celdas habría que analizar otro nivel.

Solución de la variante B

```
matrix = [[0] * 3 for _ in range(3)]
copia = [fila.copy() for fila in matrix]
copia[0][0] = 1
print(matrix)
print(copia)
print(copia[0] is copia[1])
print(matrix[0] is copia[0])
```

Salida

```
[[0, 0, 0], [0, 0, 0], [0, 0, 0]]
[[1, 0, 0], [0, 0, 0], [0, 0, 0]]
False
False
```

Cada fila se construye por separado y después se copia por separado. Como sus elementos son enteros, este nivel de copia permite aislar las sustituciones de celdas.

Archivo de referencia: soluciones/s03.py.

¿Qué cambié y qué evidencia demuestra que mi solución cumple la consigna?

LECTURA Y RAZONAMIENTO

1.9 Copia profunda

Meta: Elegir el nivel de copia necesario y comprobar la independencia entre objetos mutables originales y copiados.

Si necesito modificar listas interiores sin afectar al original, ¿qué debo copiar?

El módulo `copy` forma parte de la biblioteca estándar. `copy.copy(objeto)` realiza una copia superficial; `copy.deepcopy(objeto)` recorre y copia recursivamente los componentes que requieren copia. Con nuestras listas y diccionarios anidados, esto permite separar también los contenedores interiores.

La elección depende del cambio previsto. Una asignación sirve para compartir el mismo objeto. Una copia superficial basta para cambiar por separado las posiciones del exterior. Una copia profunda permite modificar contenedores interiores de estos ejemplos sin afectar los originales.

Ejemplo resuelto D09

```
import copy

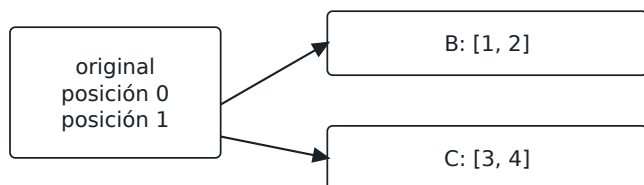
original = [[1, 2], [3, 4]]
profunda = copy.deepcopy(original)
profunda[0][0] = 99
print(original)
print(profunda)
print(original[0] is profunda[0])
```

Salida

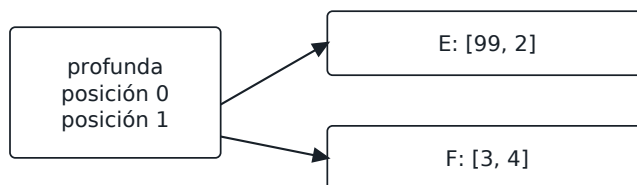
```
[[1, 2], [3, 4]]
[[99, 2], [3, 4]]
False
```

Se crea otro exterior y se copian las filas. `profunda[0]` y `original[0]` son listas diferentes; modificar la primera no cambia la segunda. No hace falta que todos los objetos inmutables tengan otra identidad para lograr esta independencia.

Original



Copia profunda, tras cambiar una celda



Observa: `deepcopy` no equivale a crear una copia distinta por cada aparición: puede conservar las relaciones de referencia compartida dentro de la copia. Para construir filas independientes, corrige su construcción; no confíes en `deepcopy` para separar alias internos.

RESUELVE EN PAPEL

1.9 Predice y explica

Escribe la salida antes de ejecutar. Conserva tu primera respuesta y usa cajas y flechas para justificarla. Los programas se reinician de forma independiente.

E17 | Una lista dentro de un diccionario

```
import copy

original = {"notas": [8, 9]}
copia = copy.deepcopy(original)
copia["notas"].append(10)
print(original["notas"])
print(copia["notas"])
```

1. Predice la salida.
2. ¿Qué objetos mutables se necesitan copiar para aislar este cambio?

Predicción, dibujo o explicación:

E18 | Una copia profunda conserva un alias interno

```
import copy

fila = [0]
original = [fila, fila]
copia = copy.deepcopy(original)
copia[0][0] = 7
print(original)
print(copia)
print(copia[0] is copia[1])
print(copia[0] is fila)
```

1. Predice la salida y dibuja originales y copias.
2. Distingue independencia respecto al original e independencia entre las filas de copia.

Predicción, dibujo o explicación:

Una variación

En e17 usa `original.copy()`. En e18, crea `copia = [elemento.copy() for elemento in original]`. Predice qué cambia en cada caso.

Predicción de la variación y motivo:

RESPUESTAS RAZONADAS | DESPUÉS DEL INTENTO

1.9 Comprueba tu razonamiento

Compara tus respuestas con las explicaciones. Si te equivocaste, conserva tu predicción inicial e identifica qué objeto o referencia interpretaste de otra manera.

E17 | Salida esperada

```
[8, 9]
[8, 9, 10]
```

La lista notas de la copia es distinta de la original. `append(10)` solo modifica la lista copiada. Una copia superficial del diccionario conservaría compartida esa lista.

E18 | Salida esperada

```
[[0], [0]]
[[7], [7]]
True
False
```

`deepcopy` crea una nueva fila y reutiliza esa copia para las dos referencias internas. La fila original permanece intacta. Hay independencia respecto al original, pero las dos posiciones de copia continúan compartiendo una fila entre sí.

La variación

En e17 ambas listas mostrarían `[8, 9, 10]`: la lista interior se comparte. En e18 las salidas serían `[[0], [0]]`, `[[7], [0]]`, `False` y `False`. Cada iteración hace una copia diferente de la fila; eso basta porque sus elementos son enteros.

Antes de avanzar

Puedo justificar el nivel de copia elegido y distinguir aislamiento del original de separación de alias dentro de una copia.

Marca tu situación: ☐ Puedo justificarlo. ☐ Necesito revisar un ejemplo.

Mi corrección o explicación con mis propias palabras:

Si la explicación aún no es clara, vuelve al diagrama del tema y sigue cada flecha desde la instrucción que produce el cambio.

LECTURA Y RAZONAMIENTO

1.10 Integración y matices de mutabilidad

Meta: Resolver casos que combinan contenedores, funciones y copias, identificando con precisión el objeto que cambia.

¿Puede cambiar una lista contenida en una tupla aunque la tupla sea inmutable?

Una tupla mantiene las referencias de sus posiciones. Eso impide sustituir sus elementos, pero no vuelve inmutables a los objetos referidos. Si contiene una lista, esa lista conserva su mutabilidad y sus cambios pueden observarse al imprimir la tupla.

En problemas con varios niveles, sigue la ruta completa hasta el objeto afectado. En `carro["motor"]["caballos"] = 300` se modifica el diccionario motor. Copiar solo el diccionario carro no separa automáticamente ese motor. Los ejemplos de personas, equipos y carros se representan aquí con diccionarios.

Ejemplo resuelto D10

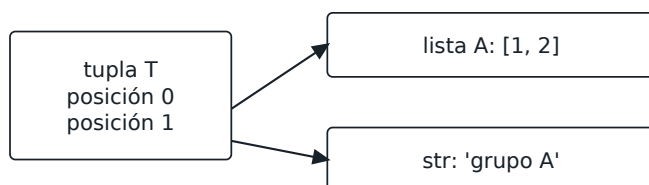
```
datos = ([1, 2], "grupo A")
alias = datos
datos[0].append(3)
print(datos)
print(alias is datos)
```

Salida

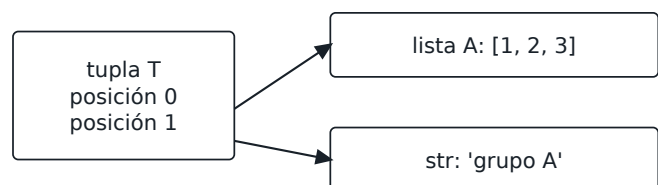
```
([1, 2, 3], 'grupo A')
True
```

La tupla sigue conteniendo las mismas referencias: una a la lista y otra a la cadena. Lo que cambia es la lista. Una asignación `datos[0] = [9]` intentaría sustituir una posición de la tupla y produciría `TypeError`.

Antes de `append(3)`



Después: mismas referencias en T



Observa: Identifica siempre el objeto modificado. 'Hay una tupla' no significa que todo lo alcanzable sea inmutable, y 'hay una copia' no significa que todo lo alcanzable sea independiente.

RESUELVE EN PAPEL

1.10 Predice y explica

Escribe la salida antes de ejecutar. Conserva tu primera respuesta y usa cajas y flechas para justificarla. Los programas se reinician de forma independiente.

E19 | Copiar una lista que contiene una tupla

```
original = [[1], "A"]
copia = original.copy()
copia[0][0].append(2)
print(original)
print(copia)
print(original[0] is copia[0])
```

1. Predice las salidas y dibuja los tres niveles.
2. ¿Qué objeto cambia y cuáles permanecen compartidos?

Predicción, dibujo o explicación:

E20 | Dos carros comparten motor

```
motor = {"caballos": 200}
carro1 = {"marca": "Toyota", "motor": motor}
carro2 = carro1.copy()
carro2["marca"] = "Honda"
carro2["motor"]["caballos"] = 300
print(carro1["marca"])
print(carro1["motor"]["caballos"])
print(carro1["motor"] is carro2["motor"])
```

1. Predice las salidas y explica por qué marca y motor se comportan de forma distinta.
2. Propón cómo conservar el motor de carro1 sin cambios.

Predicción, dibujo o explicación:

Una variación

En e20 usa `copy.deepcopy(carro1)` después de importar `copy`. En d10, explica por qué `datos[0].append(3)` y `datos[0] = [3]` afectan objetos diferentes.

Predicción de la variación y motivo:

COMPRUEBA EN PYTHON | PRÁCTICA DE LABORATORIO

P04 Un equipo de trabajo independiente

Meta: Aplicar funciones y copia profunda a un problema con personas representadas por diccionarios.

Archivo: practicas/p04.py. Trabaja desde la carpeta etapa_01_objetos_referencias.

Programa inicial: primero predice

```
def preparar_equipo(equipo):
    nuevo = equipo.copy()
    nuevo["nombre"] = "Pruebas"
    nuevo["miembros"][0]["edad"] = 99
    return nuevo

ana = {"nombre": "Ana", "edad": 25}
original = {"nombre": "Desarrollo", "miembros": [ana]}
copia = preparar_equipo(original)
print(original["nombre"], ana["edad"])
print(copia["nombre"], copia["miembros"][0]["edad"])
```

- 1. Antes de ejecutar, dibuja equipo, la lista miembros y el diccionario de Ana. Predice las dos salidas.
- 2. Corrige preparar_equipo para que el equipo devuelto pueda cambiar de nombre y edad sin alterar original ni ana.
- 3. Comprueba tres niveles con is: equipo exterior, lista miembros y primer diccionario de persona. Justifica cada comparación.
- 4. Modifica después la edad de Ana en el original y verifica que la copia conserva su propia edad. Explica qué garantía comprobaste.

Registro de comprobación

Puedes ampliar este registro en tu cuaderno. Cada variante se ejecuta desde el estado inicial.

Caso	Predicción	Observación y explicación
Inicial		
Variante / corrección		

Para ejecutar: python3 practicas/p04.py | En Windows: py practicas/p04.py

RESPUESTAS RAZONADAS | DESPUÉS DEL INTENTO

1.10 Comprueba tu razonamiento

Compara tus respuestas con las explicaciones. Si te equivocaste, conserva tu predicción inicial e identifica qué objeto o referencia interpretaste de otra manera.

E19 | Salida esperada

```
[[[1, 2], 'A']]  
[[[1, 2], 'A']]  
True
```

El exterior se copia, pero su elemento es la misma tupla. La tupla conduce a la misma lista interior, que recibe el 2. La mutabilidad de ese objeto interior sigue siendo relevante aunque el elemento inmediato del exterior sea una tupla.

E20 | Salida esperada

```
Toyota  
300  
True
```

Los diccionarios exteriores son distintos: cambiar la marca de carro2 no cambia la de carro1. El motor sigue compartido y pasa a contener 300 caballos. La última comparación comprueba esa identidad compartida.

La variación

Con deepcopy, e20 imprimiría Toyota, 200 y False: se separa también el diccionario motor. append(3) modifica la lista interior de d10; la asignación datos[0] = [3] intentaría modificar las posiciones de la tupla y fallaría con TypeError.

Antes de avanzar

Puedo explicar y corregir un cambio compartido inesperado en un programa con listas y diccionarios anidados, sin depender de probar al azar.

Marca tu situación: [] Puedo justificarlo. [] Necesito revisar un ejemplo.

Mi corrección o explicación con mis propias palabras:

SOLUCIÓN DE REFERENCIA | DESPUÉS DEL INTENTO

P04 Revisión de la práctica

Inicial: Desarrollo 99 y Pruebas 99. Solo se separó el diccionario exterior. El nombre de equipo no se propaga, pero la edad del diccionario compartido sí.

La solución imprime Desarrollo 25 y Pruebas 99. Las tres comparaciones de identidad son False porque se copiaron el exterior, miembros y el diccionario de persona.

Si después ejecutas `ana["edad"] = 26`, original observa 26 y copia conserva 99. Es una comprobación adicional de que las personas son diccionarios independientes.

Una solución con copia profunda

```
import copy

def preparar_equipo(equipo):
    nuevo = copy.deepcopy(equipo)
    nuevo["nombre"] = "Pruebas"
    nuevo["miembros"][0]["edad"] = 99
    return nuevo

ana = {"nombre": "Ana", "edad": 25}
original = {"nombre": "Desarrollo", "miembros": [ana]}
copia = preparar_equipo(original)
print(original["nombre"], ana["edad"])
print(copia["nombre"], copia["miembros"][0]["edad"])
print(original is copia)
print(original["miembros"] is copia["miembros"])
print(ana is copia["miembros"][0])
```

Salida

```
Desarrollo 25
Pruebas 99
False
False
False
```

`deepcopy` separa los tres niveles mutables. La función devuelve el exterior nuevo y el llamador lo conserva en copia.

Archivo de referencia: `soluciones/s04.py`.

¿Qué cambié y qué evidencia demuestra que mi solución cumple la consigna?

CIERRE DE LA ETAPA

Demuestra tu comprensión

Vuelve a un ejercicio que inicialmente resolviste mal. Sin mirar su respuesta, explica ahora el recorrido de sus referencias y el objeto afectado por cada instrucción.

Puedo...	Sí / Revisar
Distinguir objetos mutables e inmutables y explicar una reasignación.	
Anticipar coincidencias de identidad sin usar direcciones fijas.	
Separar la mutación de una lista de la reasignación de un nombre.	
Explicar los efectos de una función sobre el objeto recibido.	
Seguir referencias por varios niveles de contenedores.	
Construir filas independientes y comprobarlo.	
Elegir una copia superficial o profunda según el cambio previsto.	
Explicar una lista mutable dentro de una tupla inmutable.	

Una explicación completa contiene

1. Los objetos relevantes y los nombres o posiciones que llegan a ellos.
2. La instrucción que produce el cambio.
3. El objeto o referencia que se modifica.
4. El motivo por el que el resultado coincide o difiere de la predicción.

Transferencia

Elige la práctica P03 o P04. Explica qué cambiarías si el problema necesitara compartir ciertos objetos de manera intencional. Especifica cuáles compartirías y qué modificaciones serían observables desde cada referencia.

Mi explicación y mi decisión de diseño:

Puedes necesitar varios intentos. Escribe por qué tu respuesta cambió y pide apoyo señalando la instrucción o flecha que no logras explicar. El número de aciertos por sí solo no describe tu comprensión.

Conserva la guía como referencia para las siguientes etapas del curso. Esta etapa se centra en los objetos incorporados de Python y sus referencias.

CONSULTA

Glosario y fuentes

Término	Significado en esta guía
Objeto	Dato con tipo, valor e identidad.
Referencia	Relación que permite llegar a un objeto desde un nombre o un contenedor.
Alias	Otro nombre o referencia que llega al mismo objeto.
Mutación	Cambio del estado de un objeto existente.
Reasignación	Cambio del objeto asociado a un nombre.
Identidad	Distinción de un objeto frente a otros; se compara con is.
Copia superficial	Nuevo contenedor exterior que reutiliza referencias a sus elementos.
Copia profunda	Copia recursiva que puede conservar alias internos y reutilizar inmutables.

Documentación para ampliar

1. Python: objetos, valores y tipos

Consulta para 1.1-1.3 y 1.10: mutabilidad, identidad y contenedores.

<https://docs.python.org/3/reference/datamodel.html#objects-values-and-types>

2. Python: asignación de argumentos

Consulta para 1.5: asociación de parámetros y uso de valores devueltos.

<https://docs.python.org/3/faq/programming.html#how-do-i-write-a-function-with-output-parameters-call-by-reference>

3. Python: copia superficial y profunda

Consulta para 1.6, 1.7 y 1.9: operaciones del módulo copy.

<https://docs.python.org/3/library/copy.html>

4. Python: operaciones de secuencias

Consulta para 1.8: repetición de referencias y construcción de listas.

<https://docs.python.org/3/library/stdtypes.html#common-sequence-operations>

5. Raspberry Pi Foundation: PRIMM

Referencia del diseño de actividades: predecir, ejecutar, investigar, modificar y crear.

https://static.raspberrypi.org/files/curriculum/quickreads/11-Pedagogy_Summary_PRIMM_US_PRIMM.pdf

Selección y adaptación: los casos de listas, funciones, matrices, copias y equipos parten del material de referencia aportado para este curso. Las explicaciones, la progresión, los ejercicios adicionales y los diagramas se elaboraron para esta guía. Las referencias externas son de consulta; no necesitas abrirlas para realizar las actividades.